

CMSC 471 — Programming Assignment 1

Written Analysis (pa01.pdf)
Saumya Brahmabhatt | Section 03, MW 10:00–11:15

Question 1: Heuristic design and admissibility

All three heuristics share a common core. Let $\text{count}(n)$ be the number of positions at which the word in node n differs from the goal word. Because every legal action changes exactly one position, any solution path from n to the goal must contain at least $\text{count}(n)$ actions. Each of the three heuristics below lower-bounds the cost of those unavoidable actions under its own cost measure. All three return 0 at the goal, since $\text{count}(\text{goal}) = 0$.

1. Steps

Formula. $h(n) = \text{count}(n) = |\{ i : n[i] \neq \text{goal}[i] \}|$

What it lower-bounds. Under the steps measure every action costs exactly 1, so the true remaining cost is exactly the number of remaining actions. Each of the $\text{count}(n)$ mismatched positions must be corrected by at least one action of its own, because a single action rewrites only one position.

Why it cannot overestimate. The optimal remaining cost equals the number of actions on an optimal remaining path, and that path must contain at least $\text{count}(n)$ actions. Any real path may need more, since it may have to pass through detour words that change an already-correct position and then change it back, but it can never need fewer. So $h(n) \leq h^*(n)$.

2. Scrabble

Formula. $h(n) = \sum \text{SCRABBLE_VALUES}[\text{goal}[i]]$ over all i where $n[i] \neq \text{goal}[i]$

What it lower-bounds. Under the Scrabble measure an action's cost is the tile value of the letter being inserted. For each mismatched position i , the letter $\text{goal}[i]$ must be written into position i at some point on the path, and the action that finally does so costs exactly $\text{SCRABBLE_VALUES}[\text{goal}[i]]$.

Why it cannot overestimate. These charges are incurred by distinct actions: one action writes into exactly one position, so no single action can be the final placement for two different mismatched positions. Summing them therefore counts each charge at most once and never double-counts. A real path may additionally pay for detour letters, which only increases its cost. So $h(n) \leq h^*(n)$.

3. Frequency

Formula. $h(n) = 0$ if $\text{count}(n) = 0$, otherwise $h(n) = \text{count}(n) + \text{rarity}(\text{goal})$

What it lower-bounds. Under the frequency measure entering word w costs $1 + \text{rarity}(w)$, where rarity is nonnegative. Any non-goal node still needs at least $\text{count}(n)$ actions, each contributing at least 1. Separately, the very last action on any path must enter the goal word itself, and that action costs $1 + \text{rarity}(\text{goal})$.

Why it cannot overestimate. Split the remaining path into its final action and everything before it. The final action costs $1 + \text{rarity}(\text{goal})$. The other actions number at least $\text{count}(n) - 1$ and each costs at least 1, since rarity is nonnegative. Adding these gives a lower bound of $(\text{count}(n) - 1) + 1 + \text{rarity}(\text{goal}) = \text{count}(n) + \text{rarity}(\text{goal})$, which is exactly $h(n)$. The rarity of the intermediate words is discarded, which is what keeps

the bound safe. The guard for $\text{count}(n) = 0$ ensures h returns 0 at the goal rather than $\text{rarity}(\text{goal})$.

Question 2: Does the heuristic actually help?

Both searches were run with `display=True` on a fresh DC instance each time, and the reported expansion counts are used as the node measure.

Word pair	Cost measure	UCS cost	UCS nodes	A* cost	A* nodes	Reduction
WHY → ASK	Steps	5.00	773	5.00	16	97.93%
WHY → ASK	Scrabble	9.00	756	9.00	13	98.28%
WHY → ASK	Frequency	38.86	808	38.86	634	21.53%
BEAR → DUCK	Steps	4.00	691	4.00	6	99.13%
BEAR → DUCK	Scrabble	11.00	2952	11.00	6	99.80%
BEAR → DUCK	Frequency	38.24	1084	38.24	302	72.14%

1. Did UCS and A* always return the same optimal cost?

Yes. On all six runs the two searches agreed exactly: 5.00, 9.00, 38.85541, 4.00, 11.00 and 38.24023. There was no discrepancy to investigate. This is the expected outcome, since A* with an admissible heuristic is guaranteed to return an optimal-cost solution, which is what UCS returns as well. The two searches also returned identical paths in all six cases, though this is not required, as equal-cost alternatives would be acceptable.

2. Under which cost measure did A* produce the largest percentage reduction?

Scrabble, for both word pairs: 98.28% for WHY → ASK and 99.80% for BEAR → DUCK. In both cases Scrabble edged out Steps, which gave 97.93% and 99.13%.

3. Under which cost measure was the heuristic least informative?

Frequency, for both word pairs, with reductions of only 21.53% and 72.14%.

4. Explaining the differences from the heuristic definitions.

The gap follows directly from how much of the true remaining cost each heuristic manages to capture.

Under **steps**, the heuristic is nearly exact. The true remaining cost is the number of remaining actions, and $h(n) = \text{count}(n)$ is that number whenever an optimal path exists that never disturbs an already-correct position. The only slack comes from detours. The deltas confirm this: for WHY → ASK they are $[-2, -1, -1, 0.0, 0.0]$, so the heuristic is off by at most 2 and is exact over the last two nodes. A near-exact heuristic lets A* follow essentially a single line to the goal, which is why only 16 and 6 nodes were expanded.

Under **scrabble**, the heuristic is similarly tight, because it charges the exact tile value that must eventually be paid for each mismatched position. It does slightly better than steps in relative terms because the Scrabble cost measure spreads costs unevenly across actions, so UCS is forced to explore many more equal-cost frontier nodes before committing. For BEAR → DUCK, UCS expanded 2952 nodes under Scrabble versus 691 under Steps, while A* expanded 6 under both. The heuristic's absolute performance was unchanged; UCS simply got worse, which inflates the reduction.

Under **frequency**, the heuristic is structurally much looser. The true cost of entering word w is $1 + \text{rarity}(w)$, and rarity values in this dictionary are large relative to the constant 1, with $\text{rarity}(\text{ask}) = 4.23$ and $\text{rarity}(\text{duck}) = 7.23$. The optimal solution costs are around 38, of which the rarity terms are the dominant part. But the heuristic only ever charges the rarity of the goal word and discards the rarity of every intermediate word on the path. That omission is exactly what makes it admissible, and it is also what makes it weak: for $\text{WHY} \rightarrow \text{ASK}$ the heuristic at the start is $3 + 4.23 = 7.23$ against a true remaining cost of 38.86, so it accounts for under a fifth of the real cost. With five-sixths of the cost invisible to the heuristic, A^* behaves much like UCS and expands 634 of the 808 nodes UCS expanded. The $\text{BEAR} \rightarrow \text{DUCK}$ case does better at 72.14% because $\text{rarity}(\text{duck}) = 7.23$ is larger, so a bigger constant is recovered, but the same limitation applies.

Question 3: Reversibility and direction-dependent costs

Every DOGCAT action is reversible: if replacing position i of w_1 with letter c yields the legal word w_2 , then replacing position i of w_2 with the original letter yields w_1 , and both are in the dictionary. Reversibility of the action, however, says nothing about whether the action costs the same in both directions. The three cost measures differ precisely on this point.

Steps: yes, always symmetric.

Every action costs exactly 1 regardless of which letter is replaced or which letter replaces it. Given an optimal path from W_1 to W_2 with k actions, reversing the sequence gives a legal path from W_2 to W_1 with the same k actions, and hence the same cost k . The argument runs in both directions, so neither optimal cost can be smaller than the other, and they must be equal.

Scrabble: no, not always symmetric.

The cost of an action is the tile value of the letter being inserted, not of the letter being removed. Reversing an action swaps those two roles, so the reversed action generally costs a different amount.

Counterexample: $\text{pig} \rightarrow \text{dig}$ costs 2, because the inserted letter is d and $\text{SCRABBLE_VALUES}[d] = 2$. The reverse, $\text{dig} \rightarrow \text{pig}$, costs 3, because the inserted letter is p and $\text{SCRABBLE_VALUES}[p] = 3$. Both pig and dig are in the supplied dictionary and each direction is a single optimal action, so the optimal costs are 2 and 3 respectively.

Frequency: no, not always symmetric.

Entering word w costs $1 + \text{rarity}(w)$, and the initial word contributes nothing. Reversing a path therefore changes which words are charged: the original start word becomes charged, and the original goal word stops being charged. The difference between the two directions is exactly $\text{rarity}(\text{start}) - \text{rarity}(\text{goal})$.

Counterexample: $\text{dog} \rightarrow \text{cat}$ has optimal cost 23.02 along $\text{dog} \rightarrow \text{don} \rightarrow \text{dan} \rightarrow \text{can} \rightarrow \text{cat}$, while $\text{cat} \rightarrow \text{dog}$ has optimal cost 21.96 along the reverse path $\text{cat} \rightarrow \text{can} \rightarrow \text{dan} \rightarrow \text{don} \rightarrow \text{dog}$. The two paths use the same intermediate words and the same number of actions, yet differ in cost by $\text{rarity}(\text{cat}) - \text{rarity}(\text{dog}) \approx 1.06$.

Question 4: LLM heuristic audit

1. Model and prompt.

Model used: ChatGPT (GPT-4). The exact prompt given was the one specified in the assignment:

In DOGCAT, the initial and goal states are lowercase words of the same length - three or four letters - and every legal state must appear in the

supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

Proposed heuristic. The model proposed scaling the baseline by the smallest possible per-action cost. It defined $r_{\min}$ as the minimum rarity value across all words in the dictionary, and argued that since every action costs at least $1 + r_{\min}$, multiplying h_0 by that factor stays below the true cost while being strictly larger than h_0 .

2. Formula.

$$r_{\min} = \min(\text{rarity}(w) \text{ for } w \text{ in dictionary})$$
$$h_R(n) = h_0(n) * (1 + r_{\min})$$

where $h_0(n)$ = number of positions where n differs from the goal

3. Admissibility.

The heuristic is admissible, and the model's argument for it is correct. Any path from n to the goal contains at least $h_0(n)$ actions, and every action enters some word w at a cost of $1 + \text{rarity}(w) \geq 1 + r_{\min}$ by the definition of $r_{\min}$. So the true remaining cost is at least $h_0(n) \times (1 + r_{\min}) = h_R(n)$. At the goal $h_0 = 0$, so $h_R = 0$ as required.

It is also more informative than h_0 in the sense the assignment defines. Measuring the dictionary directly gives $r_{\min} = 0.016226212$. Since this is strictly positive, $h_R(n) = 1.016226 \times h_0(n) > h_0(n)$ at every state where $h_0(n) > 0$, and $h_R(n) = h_0(n) = 0$ at the goal. So the proposal satisfies both required conditions.

However, the improvement is negligible in practice. Because $r_{\min}$ is close to zero, h_R is only about 1.6% larger than h_0 , while the true remaining costs are in the range of 38. The heuristic recovers a tiny fraction of a cost it is mostly blind to. The weakness is not inadmissibility but uselessness, and this is the kind of thing the model's justification does not surface: it argues correctly that the bound holds, and says nothing about whether the bound is tight enough to change the search. Measured on the two frequency problems, h_R expanded 754 nodes on WHY $\rightarrow$ ASK against 755 for h_0 , and 714 against 720 for BEAR $\rightarrow$ DUCK. That is a saving of one node and six nodes respectively.

4. Repair.

Rather than scaling h_0 by a global minimum, a better use of rarity is to charge the rarity of the one word we know for certain must be entered: the goal itself. This gives the heuristic implemented in `dc.py`:

$$\begin{array}{ll} h(n) = 0 & \text{if } h_0(n) == 0 \\ h(n) = h_0(n) + \text{rarity}(\text{goal}) & \text{otherwise} \end{array}$$

This remains admissible by the argument given in Question 1: the final action must enter the goal and costs $1 + \text{rarity}(\text{goal})$, and the preceding $h_0(n) - 1$ actions each cost at least 1. It is strictly larger than h_R whenever $\text{rarity}(\text{goal}) > r_{\min} \times h_0(n)$, which holds for both test problems, because it recovers a real rarity value from the actual dictionary rather than the smallest one in it.

5. Test results.

Heuristic	WHY → ASK cost	nodes	BEAR → DUCK cost	nodes
h0 (baseline)	38.86	755	38.24	720
h_R (ChatGPT proposal)	38.86	754	38.24	714
h0 + rarity(goal) (repair)	38.86	634	38.24	302

Conclusion. All three heuristics return the same optimal costs, which is consistent with all three being admissible. The evidence does not support adopting the model's proposal: it is correct but saves 1 node out of 755 and 6 out of 720, which is within the range of no practical effect. The repaired heuristic cuts expansions by 16% and 58% against the baseline on the same two problems, so it is the one worth using. It is worth noting that these tests can only reveal counterexamples to admissibility; passing them does not prove admissibility, which is why the argument in Question 1 is given separately.

Generative AI disclosure

System used. ChatGPT (GPT-4) for Question 4 as required by the assignment. Claude for formatting this document and for running the verification experiments described below.

What it was used for. Question 4 required prompting a generative AI system for a proposed heuristic; that exchange is reported above. Separately, Claude was used to format the tables and prose in this document, and to run the UCS/A* comparison script and the Question 4 heuristic comparison.

How the results were verified. Every number reported in this document was produced by running my own dc.py against the supplied search.py and words34.txt.gz, using uniform_cost_search(problem, display=True) and astar_search(problem, display=True) on fresh DC instances. The r_min value in Question 4 was computed directly from the supplied dictionary rather than assumed. The Question 3 counterexamples were confirmed by solving both directions with astar_search. The admissibility arguments in Questions 1 and 4 are my own reasoning and do not rest on the model's claims.

Collaboration. None.